

# Circuits logiques

## Le transistor :

L'invention du transistor en 1947 a ouvert l'ère de l'électronique pour l'humanité et a permis à l'informatique de se miniaturiser et de se démocratiser au grand public.

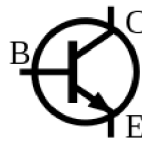
Il est possible de réaliser des opérations logiques à l'aide de transistors. En effet, chaque processeur possède dans son jeu d'instructions des opérations booléennes (ou opérations bit à bit).

**Gottfried Wilhelm Leibniz** écrit un article important en 1703 'De Progressione Dyadica'.

**Georges Boole** sort 'Mathematical Analysis of Logic' en 1847, qui est une structure algébrique à seulement deux états: 0 et 1. En 1930, **Claude Shannon** démontre qu'avec des interrupteurs fermés pour 'vrai' et ouvert pour 'faux', on peut effectuer des opérations logiques en associant le nombre 1 pour vrai et 0 pour faux. Le mot 'bit' créé par **John Tukey** en 1947.

Il existe des transistors de diverses technologies (Transistor bipolaire NPN ou PNP, transistor bipolaire à grille isolée IGBT, transistor à effet de champs : MOS FET (canal N ou P) (NMOS, PMOS)ect...)

Lorsque la base est soumise à une tension, elle laisse passer le courant entre le collecteur et l'émetteur (état 1), la broche de l'émetteur passe à l'état bas en tension. Si la tension de la base est nulle le courant, ne passe pas (état 0) la broche de l'émetteur passe à l'état haut en tension.



Une variable booléenne en Python est une variable de vérité pouvant avoir 2 valeurs : True ou False

## Opérations sur les booléens :

la négation (le NON logique) , la conjonction (le ET logique) et la disjonction (le OU logique)

Le circuit non correspond à l'opérateur not (négation). Il s'applique à une seule variable.

### Le circuit Non

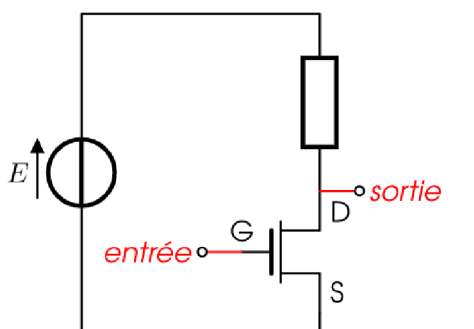


Schéma électrique

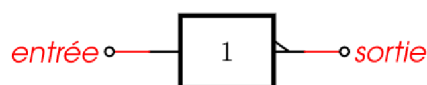
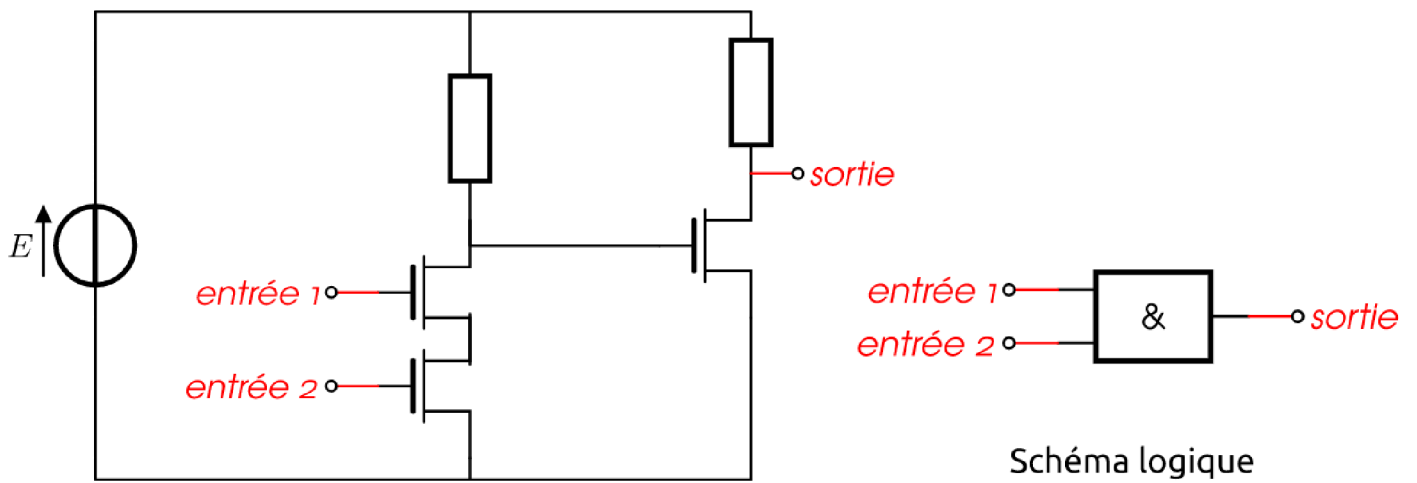


Schéma logique

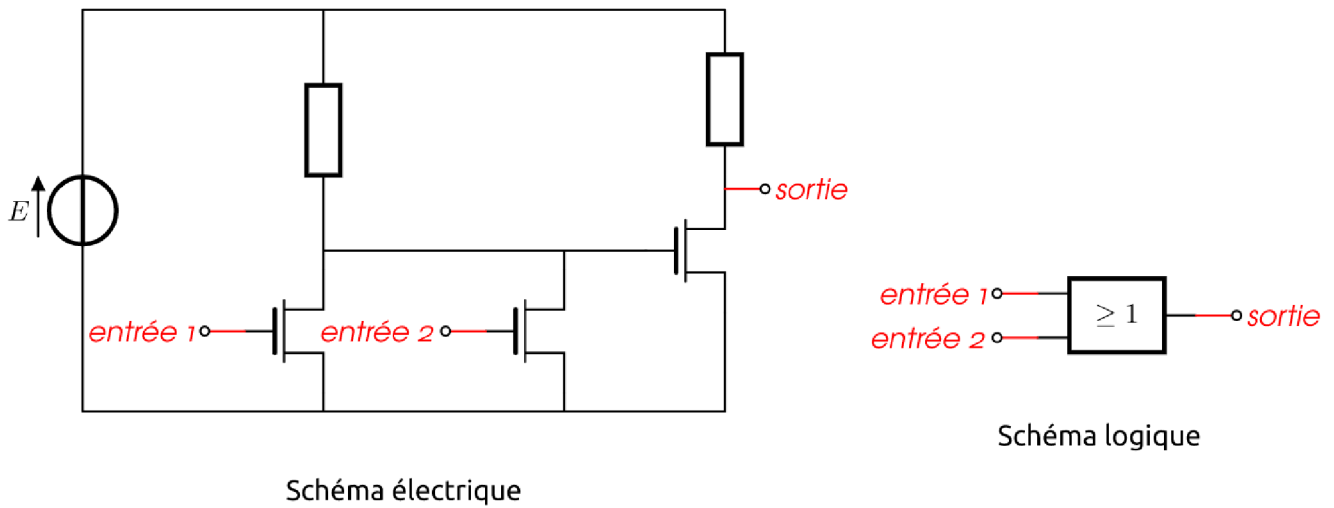
**Le circuit Et correspond à l'opérateur and :**

**Le circuit Et**



**Le circuit ou correspond à l'opérateur or :**

**Le circuit Ou**

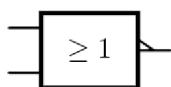


D'autres circuits et opérateurs existent en combinant les opérateurs not, and et or.

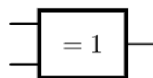
**Non-ET NAND**



**Non-OU NOR**



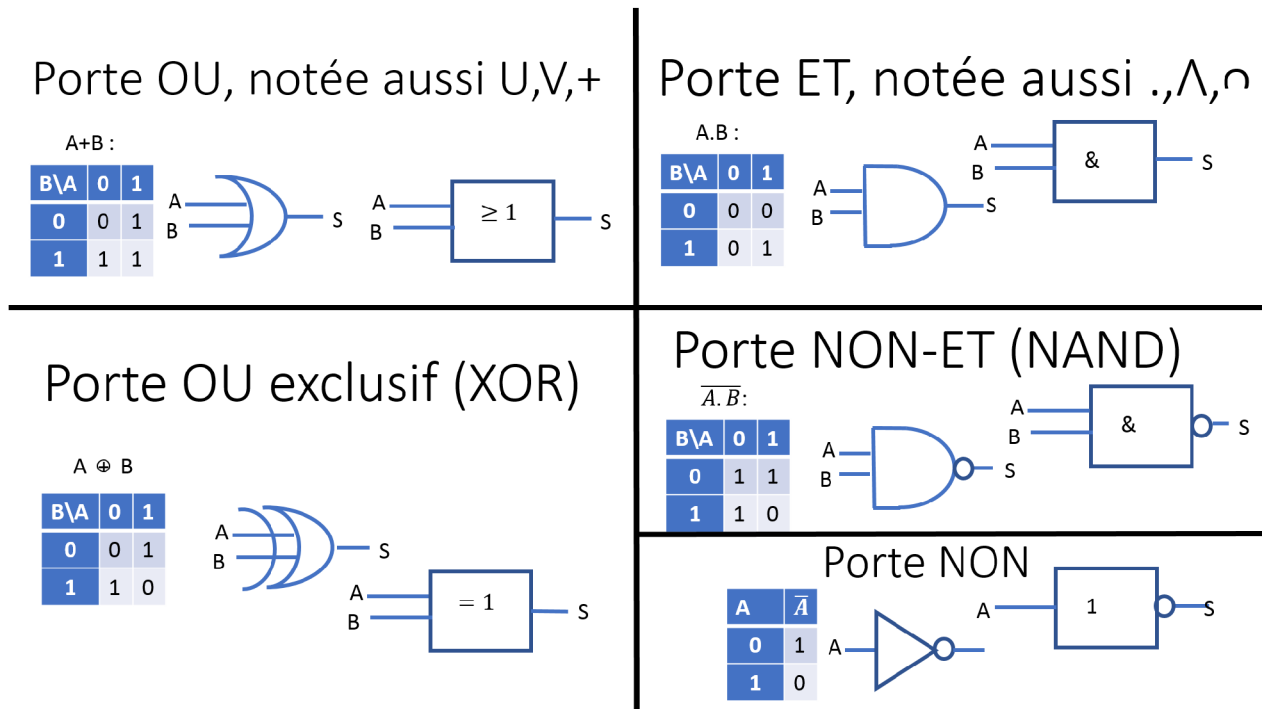
**OU eXclusif XOR**



## Table de vérité :

Une table de vérité donne tous les résultats possibles d'une opération booléenne en fonction de la ou les entrées.

Voici les tables ainsi que les autres représentations possible des portes :



## Écriture binaire d'un entier positif sur n bits :

si le nombre de bits est inférieur au nombre de bits n on complète par des 0

si le nombre de bit est insuffisant on parle d'**overflow**

exemple :

153 = (10011001)<sub>2</sub>

sur 4 bits => overflow

sur 8 bits => 10011001

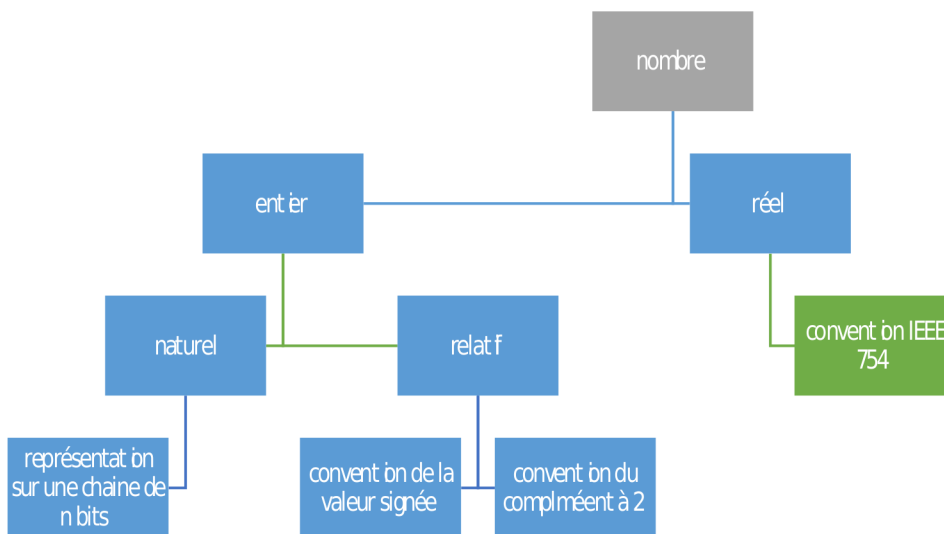
sur 16 bits => 0000000010011001

## Représentation des nombres dans une machine :

Pour représenter des nombres en machine il faut les convertir en binaire cependant une machine ou un système d'exploitation travaille avec une représentation sur un nombre de bits finis.

Lorsque nous programmons en langage Python, le nombre d'octet nécessaire pour effectuer une opération est géré automatiquement. Si un nombre dépasse la capacité maximale du processeur, Python le découpe en plusieurs morceaux et effectue les opérations supplémentaires nécessaires. La plupart des processeurs travaillent avec des nombres binaires de 32 ou 64 bits.

On utilise pour cela le schéma suivant :



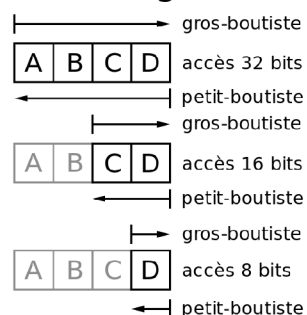
### Le petit et le gros boutisme :

Ils viennent d'une guerre de religion inventée par **Jonathan Swift** dans « Les Voyages de Gulliver ». Il s'agissait de savoir s'il faut manger les œufs par le gros bout ou par le petit bout.

En informatique, il s'agit de savoir comment on écrit les nombres en mémoire : est-ce que l'on met les bits dans l'ordre « normal » ou dans l'ordre inverse. L'ordre usuel permet une lecture par les humains plus facile, tandis que l'ordre inverse permet des calculs plus simples par le microprocesseur. Ainsi un mot de 16 bits comme AB12 sera écrit en machine AB 12 en gros boutisme (big endian) et 12 AB en petit boutisme (little endian).

Cette représentation est gérée par le système d'exploitation, on en a besoin lors d'accès d'octets en mémoire ou lors de la transmission d'informations sur le réseau.

### Registre



### Mémoire

